Chủ đề 6: Vòng đời của Activity

Tóm tắt nội dung:

- Các thành phần trong ứng dụng Android.
- Vòng đời của Activity và Intent.
- Quản lý tài nguyên và trạng thái.

Chủ đề 6: Vòng đời của Activity

Mục tiêu buổi học:

- Hiểu rõ vòng đời của Activity trong Android.
- Khám phá các thành phần cơ bản của một ứng dụng Android.
- Thực hành xử lý vòng đời ứng dụng.

Lập trình cơ bản cho ứng dụng

Khái niệm chính:

- Lập trình Android là phát triển ứng dụng di động chạy trên nền tảng Android.

- Dựa trên ngôn ngữ Kotlin hoặc Java.

Các yếu tố cơ bản:

- Giao diện người dùng (UI).

- Xử lý sự kiện người dùng.

- Quản lý dữ liệu.

Cách bắt đầu:

1. Tạo dự án mới trong Android Studio.

2. Thiết lập build.gradle với các dependencies cần thiết.

3. Bắt đầu bằng việc tạo màn hình chính (Activity).

Các thành phần cơ bản trong ứng dụng Android

Các thành phần chính:

1. Activity: Giao diện màn hình mà người dùng tương tác.

2. Service: Thành phần chạy ngầm, không có giao diện.

3. Broadcast Receiver: Nhận và xử lý các sự kiện hệ thống.

4. Content Provider: Chia sẻ dữ liệu giữa các ứng dụng.

Các thành phần cơ bản trong ứng dụng Android

Vai trò của từng thành phần: Tạo cấu trúc ứng dụng rõ ràng và dễ bảo trì.

Cách tạo một thành phần cơ bản:

1. Khai báo thành phần trong tệp AndroidManifest.xml.

2. Tạo lớp kế thừa từ thành phần cần sử dụng (e.g., AppCompatActivity, Service).

Activity: Thành phần giao diện chính

Activity là gì?

- Đại diện cho một màn hình giao diện.

- Là nơi diễn ra tương tác giữa người dùng và ứng dụng.

Vai trò của Activity:

- Hiển thị giao diện và xử lý sự kiện người dùng.

- Quản lý vòng đời ứng dụng.

Activity: Thành phần giao diện chính

Cách tạo Activity:

1. Tạo một lớp kế thừa AppCompatActivity.

2. Khai báo Activity trong AndroidManifest.xml.

3. Gắn giao diện bằng tệp XML (e.g., setContentView(R.layout.activity_main)).

Service: Thành phần chạy ngầm

Service là gì?

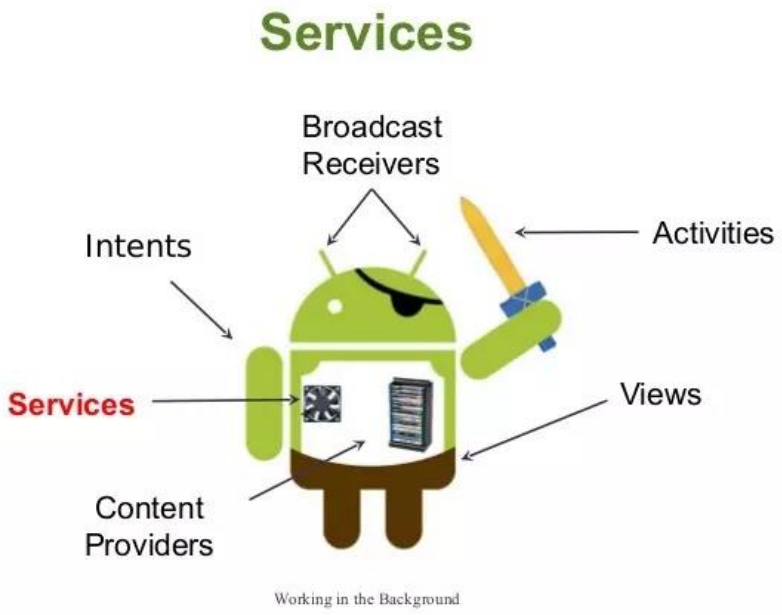
- Thực hiện các tác vụ dài hạn hoặc chạy ngầm.

- Không có giao diện người dùng.

Ví dụ sử dụng:

- Phát nhạc trong nền

- Tải tệp dữ liệu từ Internet…



Service: Thành phần chạy ngầm

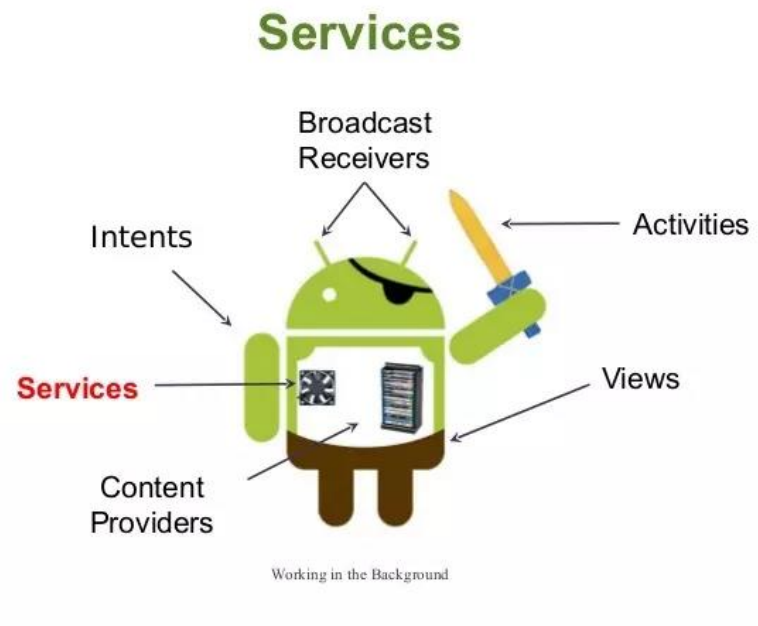
Cách tạo Service:

1. Tạo một lớp kế thừa Service hoặc IntentService.

2. Khai báo trong AndroidManifest.xml.

3. Khởi động Service bằng:

```
val intent = Intent(this, MyService::class.java)
startService(intent).
```



Service: Thành phần chạy ngầm

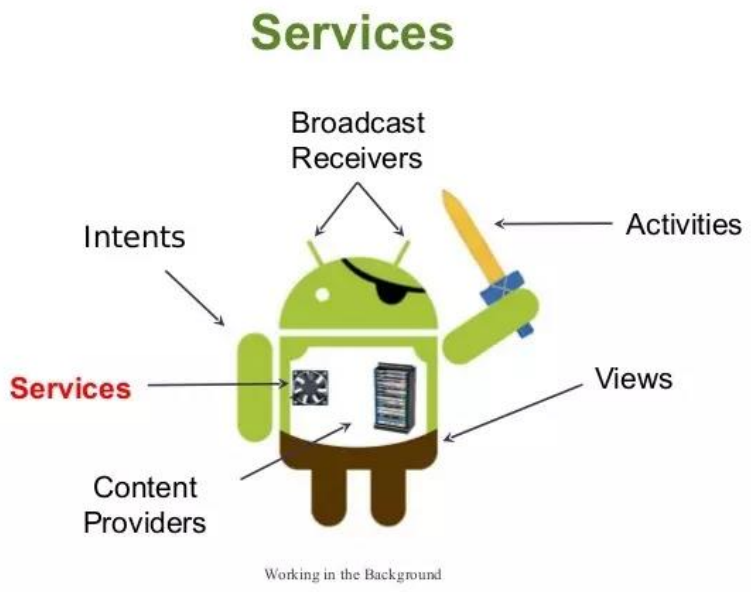
Cách tạo Service:

1. Tạo một lớp kế thừa Service hoặc IntentService.

```
class MyService : Service() {
    override fun onBind(intent: Intent?): IBinder? {
        return null // Không cần giao tiếp với Activity
    }

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        Log.d("MyService", "Service đang chạy")
        return START_STICKY
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d("MyService", "Service bị hủy")
    }
}
```



Service: Thành phần chạy ngầm

Cách tạo Service:

2. Khai báo Service trong `AndroidManifest.xml`

```
<service android:name=".MyService" />
```

3. Khởi động Service từ Activity

```
val intent = Intent(this, MyService::class.java)
startService(intent) // Bắt đầu Service
```

4. Dừng Service

```
stopService(intent) // Dừng Service
```

Ứng dụng Service trong thực tế

Tình huống thực tế:

- Ứng dụng nghe nhạc.

- Cập nhật thời tiết trong nền.

Cách triển khai:

1. Tạo một Service để xử lý phát nhạc.

2. Kết nối với Activity để điều khiển phát nhạc:

   - Sử dụng MediaPlayer để phát nhạc.

   - Liên kết Service với giao diện người dùng.

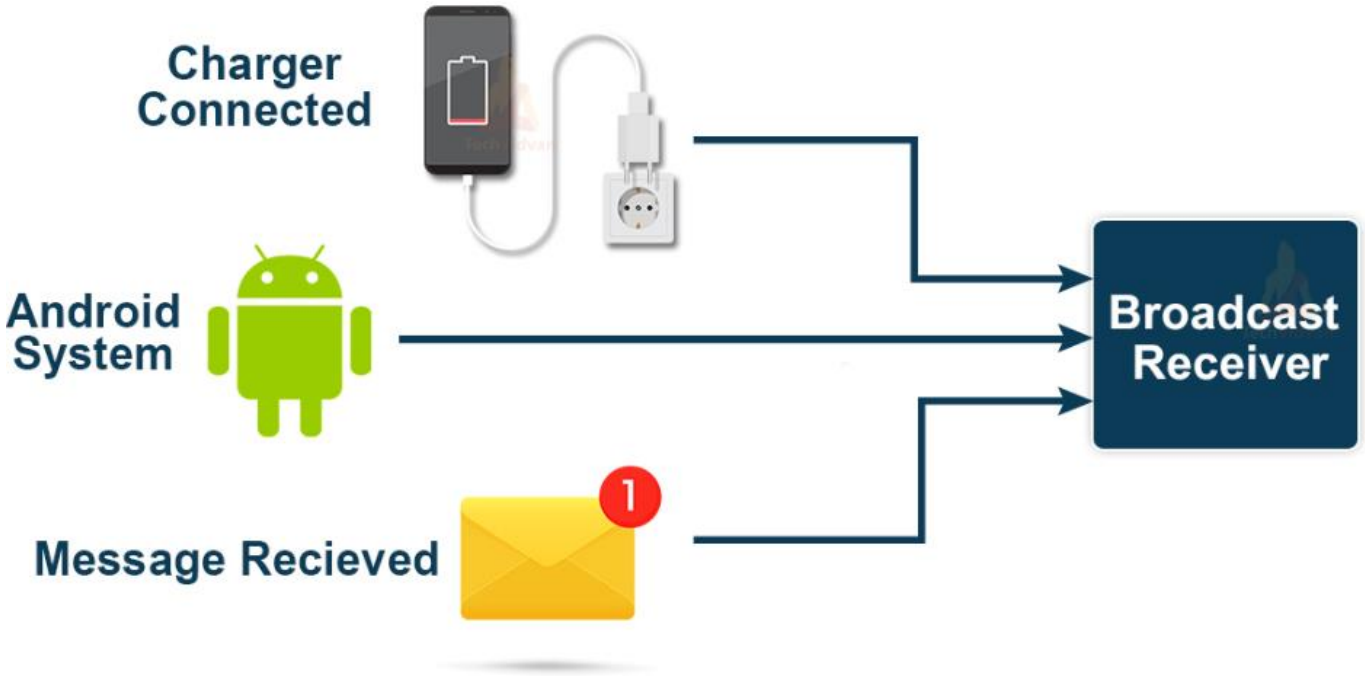
Broadcast Receiver: Xử lý sự kiện hệ thống

Broadcast Receiver là gì?

- Nhận và xử lý các sự kiện hệ thống hoặc từ ứng dụng khác.

Ví dụ:

- Pin yếu.
- Kết nối mạng thay đổi.
- Nhận tin nhắn SMS.

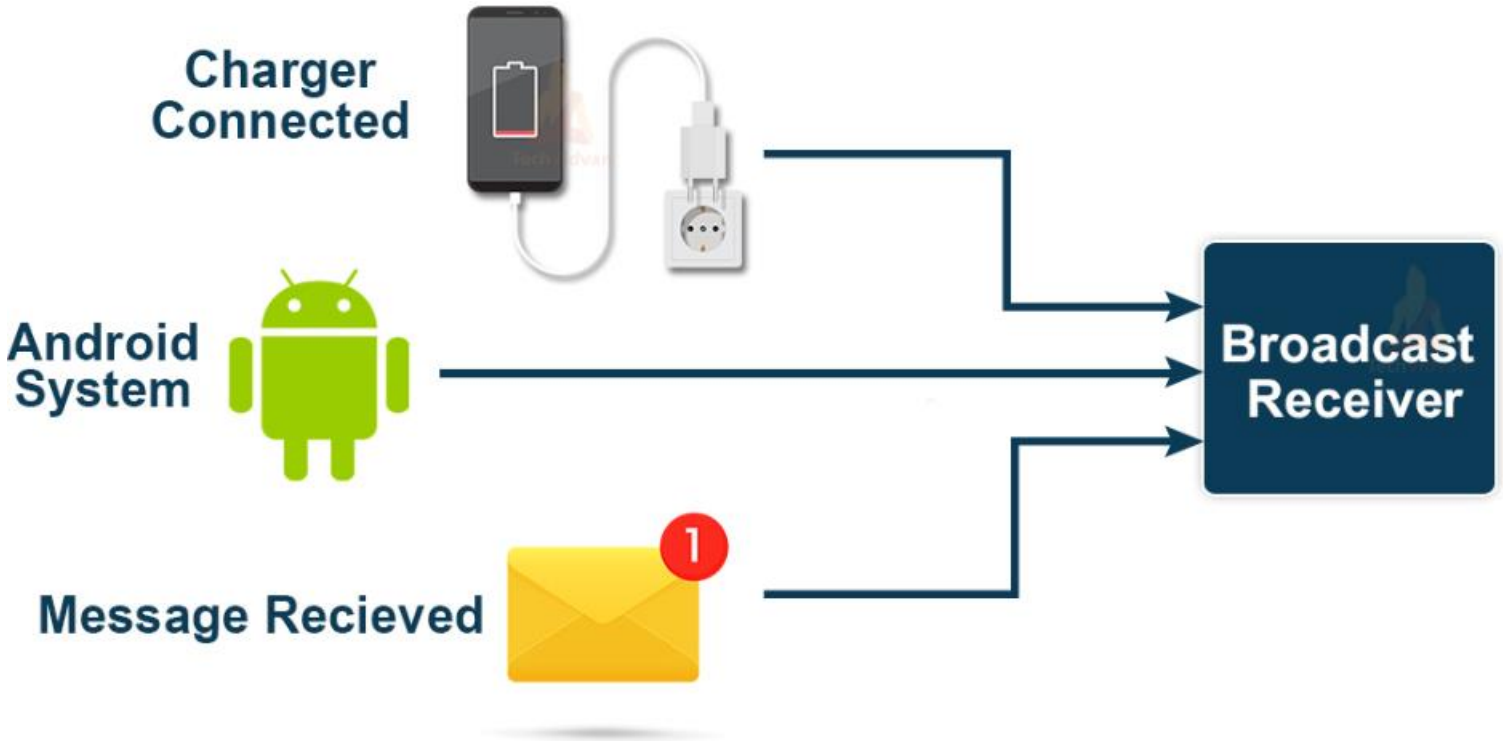


Broadcast Receiver: Xử lý sự kiện hệ thống

Cách tạo Broadcast Receiver:

1. Tạo một lớp kế thừa BroadcastReceiver.

2. Đăng ký sự kiện trong AndroidManifest.xml hoặc sử dụng registerReceiver().



Broadcast Receiver: Xử lý sự kiện hệ thống

Cách tạo Broadcast Receiver:

1. Tạo một lớp kế thừa `BroadcastReceiver`

```
class MyReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context?, intent: Intent?) {
        Log.d("MyReceiver", "Nhận được sự kiện: ${intent?.action}")
    }
}
```

Broadcast Receiver: Xử lý sự kiện hệ thống

Cách tạo Broadcast Receiver:

2. **Đăng ký trong** `AndroidManifest.xml` (Nhận sự kiện thay đổi mạng)

```
<receiver android:name=".MyReceiver">
    <intent-filter>
        <action android:name="android.net.conn.CONNECTIVITY_CHANGE" />
    </intent-filter>
</receiver>
```

3. **Đăng ký động trong Activity**

```
val receiver = MyReceiver()
val intentFilter = IntentFilter("android.net.conn.CONNECTIVITY_CHANGE")
registerReceiver(receiver, intentFilter)
```

4. **Hủy đăng ký khi không cần thiết**

```
unregisterReceiver(receiver)
```

Content Provider: Chia sẻ dữ liệu

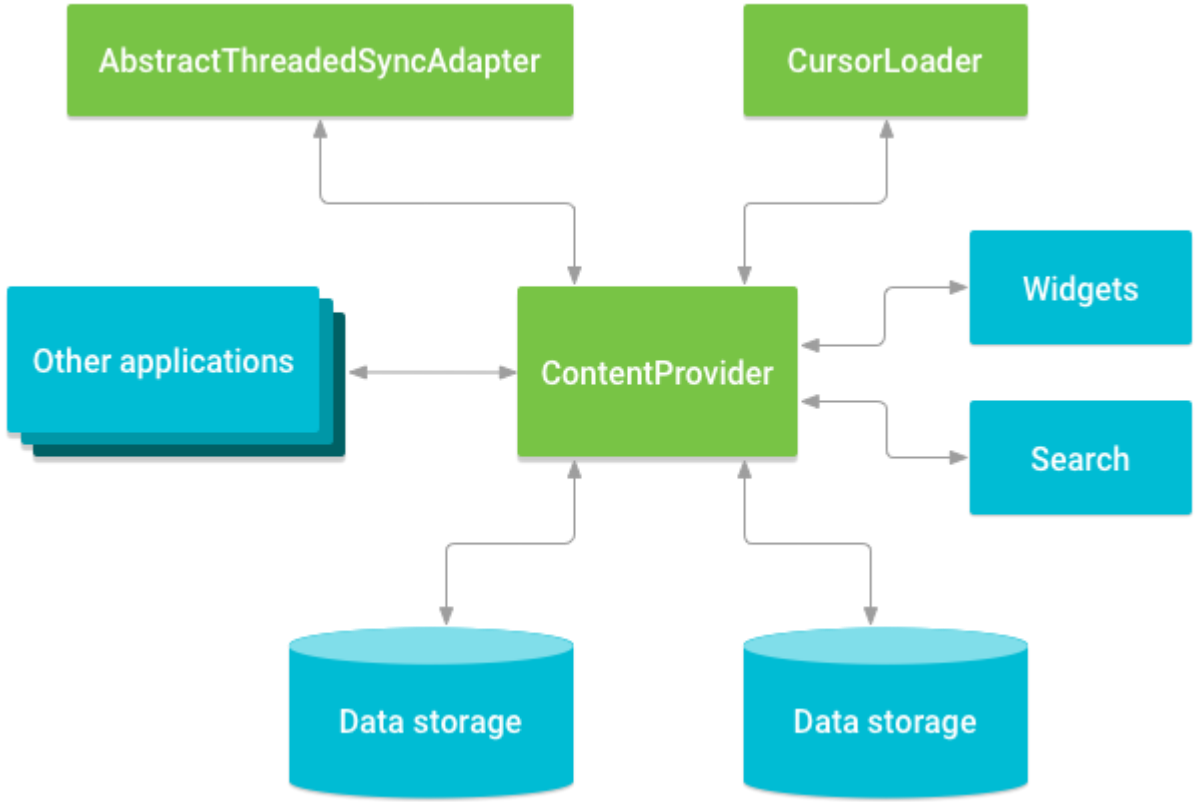
Content Provider là gì?

- Cung cấp một giao diện chuẩn để chia sẻ dữ liệu giữa các ứng dụng.

Ứng dụng thực tế:

- Truy cập danh bạ, hình ảnh.

.



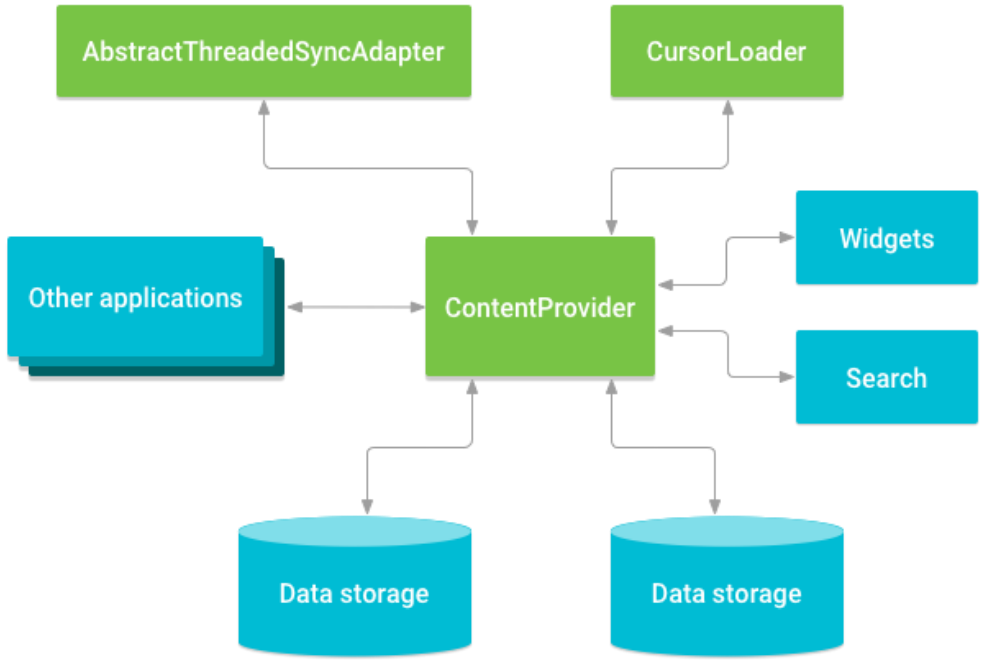
Content Provider: Chia sẻ dữ liệu

Cách tạo Content Provider:

1. Tạo lớp kế thừa ContentProvider.

2. Ghi đè các phương thức chính như query(), insert(), update(), delete().

3. Khai báo trong AndroidManifest.xml.



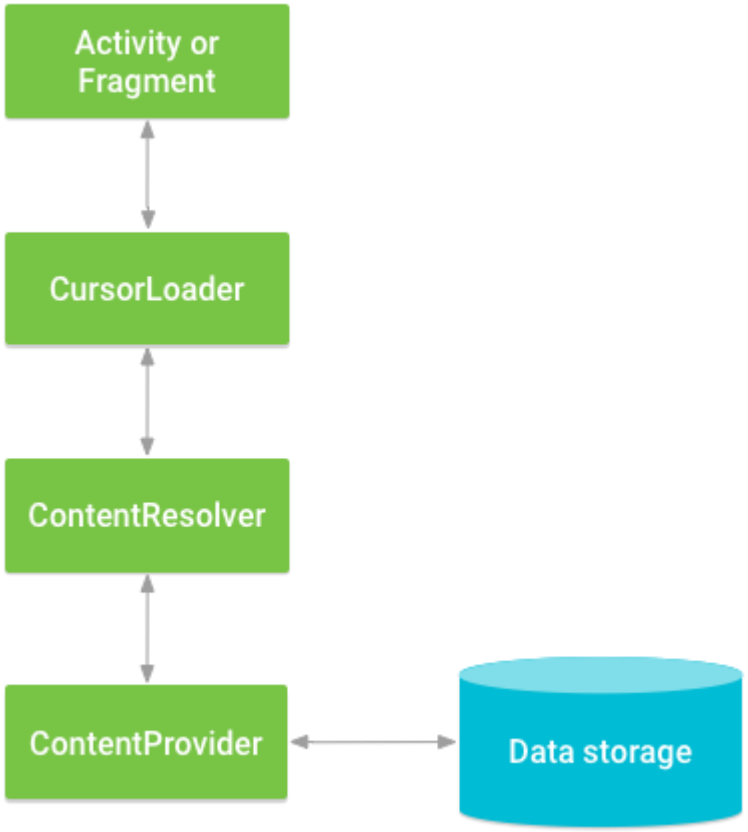
Quy trình sử dụng Content Provider

Quy trình làm việc:

1. Khởi tạo Content Provider trong ứng dụng.

2. Định nghĩa URI để truy cập dữ liệu.

3. Thực hiện các phương thức như query(), insert(), update(), delete().



Content Provider: Chia sẻ dữ liệu

Cách tạo Content Provider:

1. Tạo lớp kế thừa của lớp cha là ContentProvider.

2. Ghi đè các phương thức chính như query(), insert(), update(), delete().

```
class MyContentProvider : ContentProvider() {
    override fun onCreate(): Boolean {
        return true
    }

    override fun query(uri: Uri, projection: Array<String>?, selection: String?,
                       selectionArgs: Array<String>?,
                       sortOrder: String?): Cursor? {
        return null
    }

    override fun insert(uri: Uri, values: ContentValues?): Uri? {
        return null
    }

    override fun update(uri: Uri, values: ContentValues?,
                        selection: String?, selectionArgs: Array<String>?): Int {
        return 0
    }

    override fun delete(uri: Uri, selection: String?,
                        selectionArgs: Array<String>?): Int {
        return 0
    }

    override fun getType(uri: Uri): String? {
        return null
    }
}
```

Content Provider: Chia sẻ dữ liệu

Cách tạo Content Provider:

3. Khai báo trong AndroidManifest.xml, đăng ký sử dụng.

```
val uri = Uri.parse("content://com.example.provider/data")
val cursor = contentResolver.query(uri, null, null, null, null)
```

```
<provider
    android:name=".MyContentProvider"
    android:authorities="com.example.provider"
    android:exported="false" />
```

Quy trình sử dụng Content Provider

Ví dụ: Truy cập danh bạ người dùng

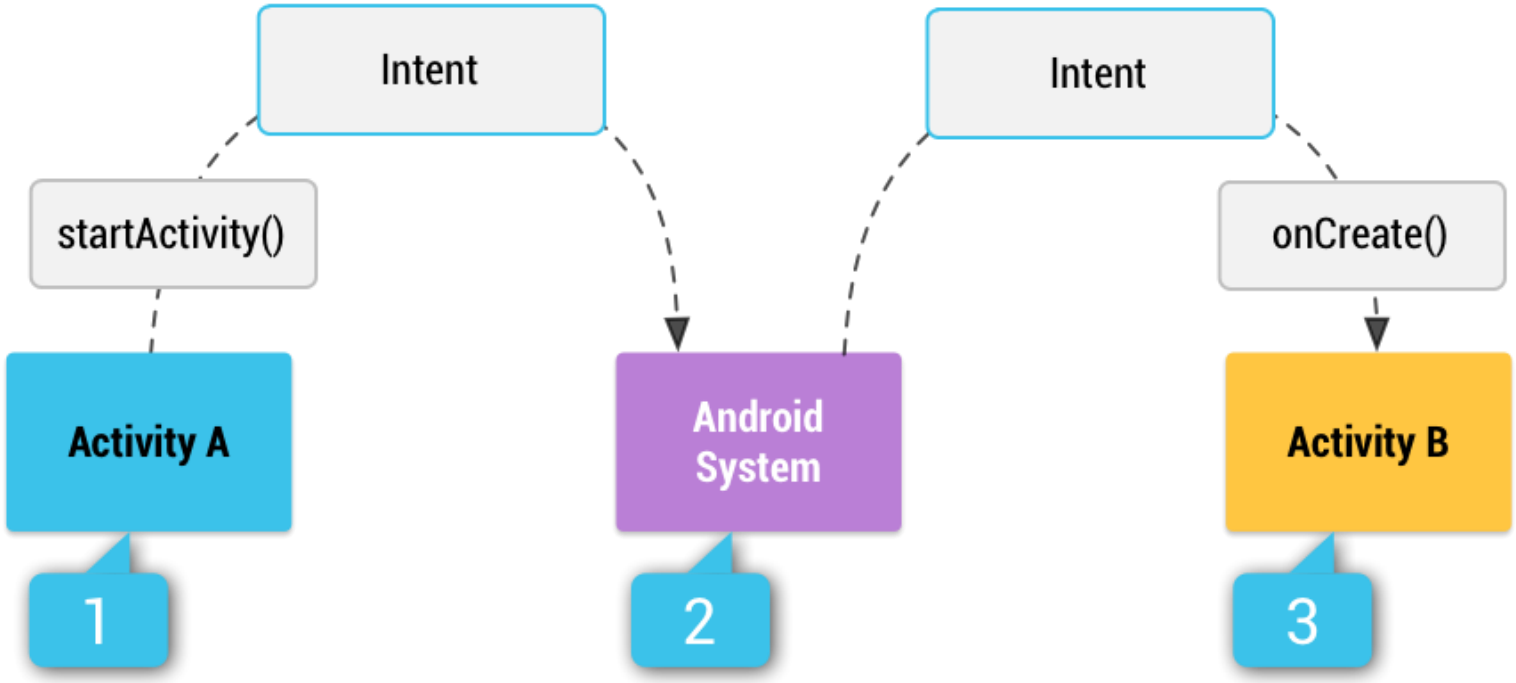
```
val uri = ContactsContract.CommonDataKinds.Phone.CONTENT_URI
val cursor = contentResolver.query(uri, null, null, null, null)
while (cursor?.moveToNext() == true) {
    val name = cursor.getString(cursor.getColumnIndex
    (ContactsContract.CommonDataKinds.Phone.DISPLAY_NAME))  }
```

Intent: Khái niệm cơ bản

Intent là gì?

- Là một đối tượng trong Android dùng để truyền dữ liệu hoặc thực hiện hành động.

- Có thể sử dụng để khởi động Activity, Service hoặc gửi Broadcast.



Intent: Khái niệm cơ bản

Vai trò của Intent:

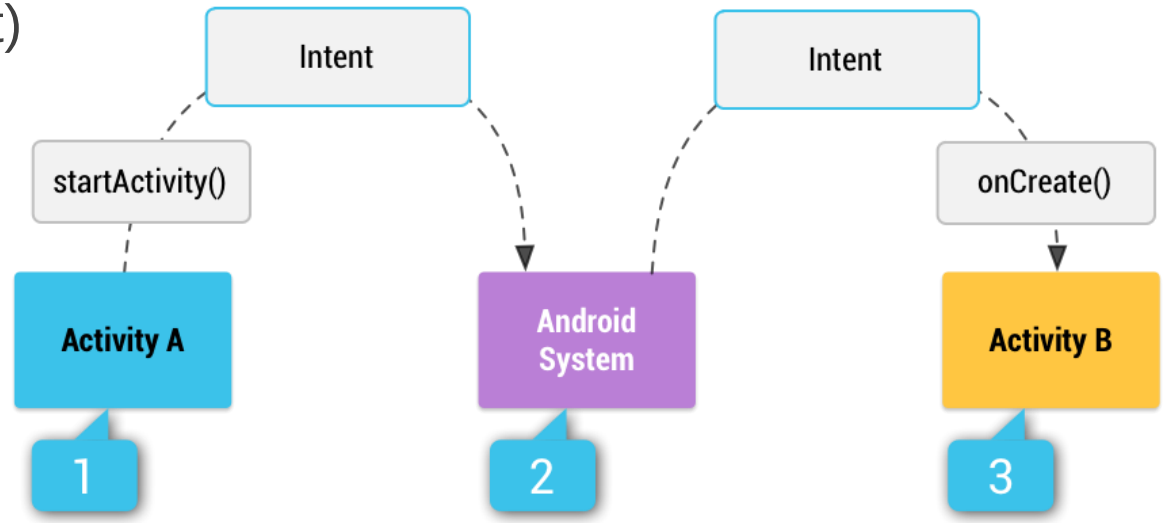
- Giao tiếp giữa các thành phần trong ứng dụng.

- Truyền dữ liệu giữa các màn hình.

Ví dụ:

```
val intent = Intent(this, SecondActivity::class.java)

startActivity(intent)
```



Phân loại Intent: Explicit và Implicit

Explicit Intent:

- Xác định rõ thành phần đích (e.g., một Activity cụ thể).
- Ví dụ:

```
val intent = Intent(this, SecondActivity::class.java)
startActivity(intent)
```

Implicit Intent:

- Không xác định rõ thành phần đích, hệ thống tìm kiếm thành phần phù hợp.
- Ví dụ:

```
val intent = Intent(Intent.ACTION_VIEW, Uri.parse("https://www.google.com"))
startActivity(intent)
```

Sử dụng Intent để khởi động Activity

Mục đích:

- Điều hướng người dùng đến một Activity khác.

Cách triển khai:

1. Tạo một Intent:

```
val intent = Intent(this, SecondActivity::class.java)
```

2. Gửi Intent:

```
startActivity(intent)
```

3. Khai báo Activity trong AndroidManifest.xml.

Chuyển dữ liệu giữa các Activity với Intent

Truyền dữ liệu:

1. Sử dụng phương thức putExtra để đính kèm dữ liệu:

```
val intent = Intent(this, SecondActivity::class.java)
intent.putExtra("USERNAME", "Nam")
startActivity(intent)
```

2. Nhận dữ liệu ở Activity đích:

```
val username = intent.getStringExtra("USERNAME")
```

Ví dụ thực tế:

- Chuyển tên người dùng từ màn hình đăng nhập sang màn hình chính.

Sự kiện hệ thống và Intent

Tương tác với hệ thống:

- Sử dụng Intent để thực hiện các hành động hệ thống như:

Gọi điện thoại:

```
val intent = Intent(Intent.ACTION_DIAL, Uri.parse("tel:123456789"))
startActivity(intent)
```

Gửi email:

```
val intent = Intent(Intent.ACTION_SEND)
intent.type = "text/plain"
intent.putExtra(Intent.EXTRA_EMAIL, arrayOf("example@gmail.com"))
intent.putExtra(Intent.EXTRA_SUBJECT, "Hello!")
startActivity(intent)
```

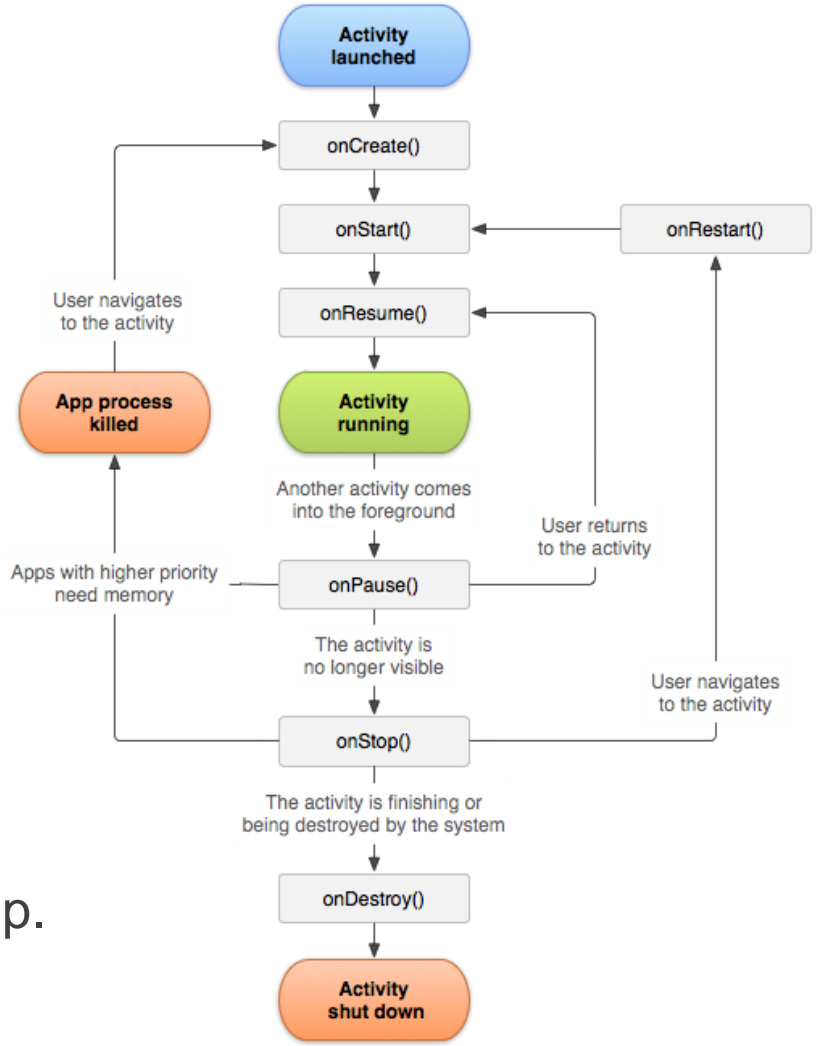
Vòng đời của Activity

Các trạng thái chính:

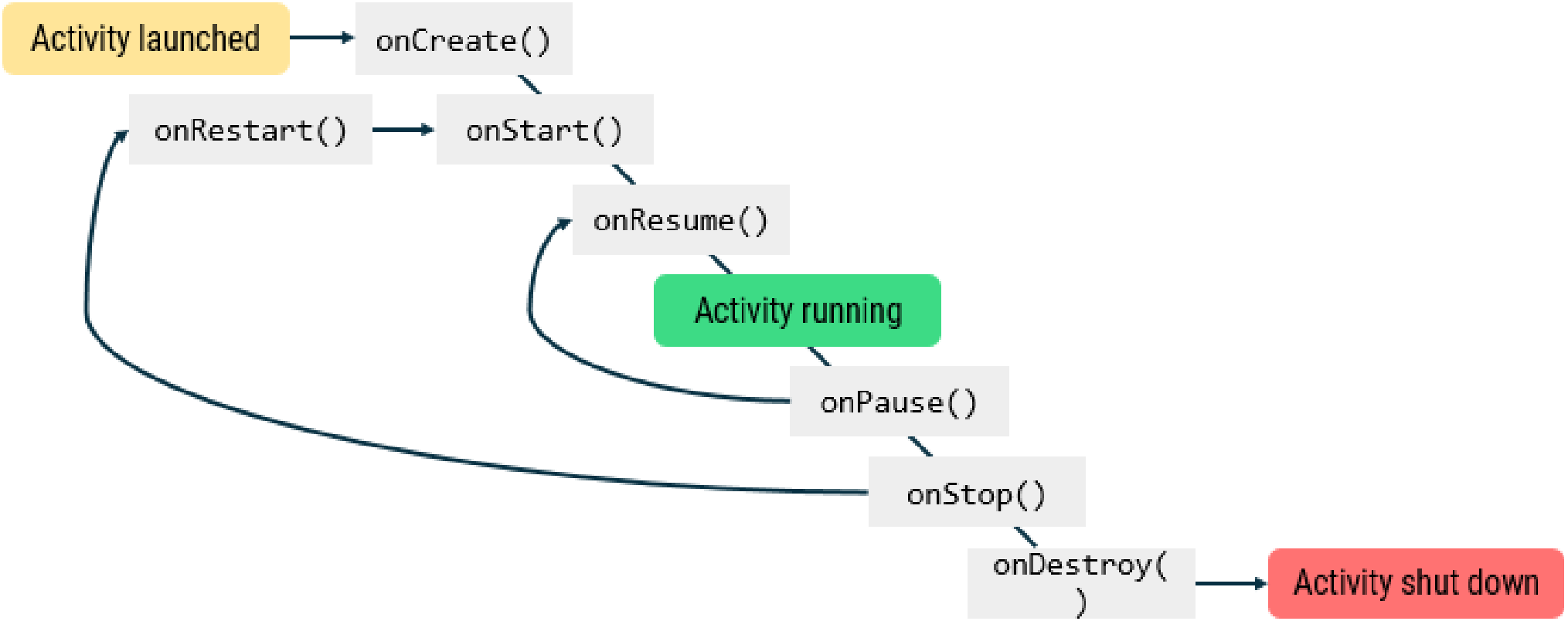
1. onCreate(): Tạo Activity.
2. onStart(): Activity bắt đầu hiển thị.
3. onResume(): Activity sẵn sàng tương tác.
4. onPause(): Tạm dừng, chuẩn bị bị ẩn.
5. onStop(): Bị ẩn hoàn toàn.
6. onDestroy(): Được hủy.

Ví dụ:

- Màn hình chính bị thay thế bởi màn hình đăng nhập.



Vòng đời của Activity



Các trạng thái trong vòng đời của Activity

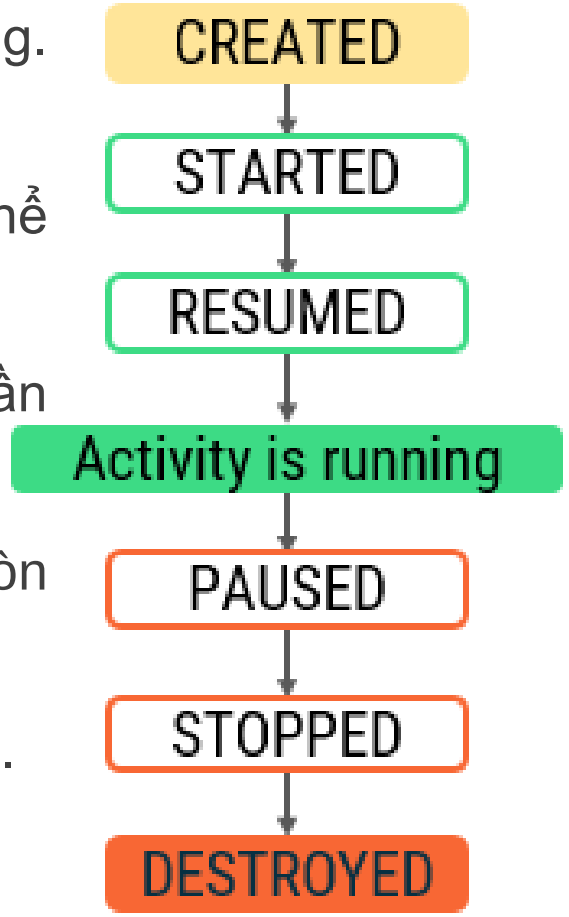
Trạng thái mô tả **tình trạng hiện tại** của Activity trong ứng dụng. Các trạng thái chính bao gồm:

Running (Hoạt động): Activity đang ở **onResume()**, có thể tương tác với người dùng.

Paused (Tạm dừng): Activity ở **onPause()**, bị che một phần nhưng vẫn chạy nền.

Stopped (Dừng hoàn toàn): Activity ở **onStop()**, không còn hiển thị trên màn hình.

Destroyed (Bị hủy): Activity bị hủy hoàn toàn, gọi **onDestroy()**.



Các trạng thái trong vòng đời của Activity

Hàm vòng đời	Trạng thái của Activity
`onCreate()`	Khởi tạo (Initializing)
`onStart()`	Hiển thị trên màn hình nhưng chưa tương tác
`onResume()`	Đang chạy, có thể tương tác với người dùng
`onPause()`	Tạm dừng, vẫn tồn tại nhưng bị che khuất
`onStop()`	Không còn hiển thị nhưng vẫn lưu trong bộ nhớ
`onDestroy()`	Bị hủy hoàn toàn, giải phóng tài nguyên

Ví dụ minh họa vòng đời

Tình huống thực tế:

- Khi người dùng bấm nút Home, ứng dụng chuyển từ onResume() → onPause() → onStop().

- Khi quay lại, chuyển từ onStop() → onRestart() → onStart() → onResume().

Thực hành:

- Tạo một ứng dụng hiển thị trạng thái vòng đời bằng Toast.

Quản lý vòng đời Activity

Tầm quan trọng:

- Đảm bảo tài nguyên được giải phóng khi không cần thiết.

- Giảm thiểu lỗi khi Activity bị tiêu hủy.

Cách quản lý:

1. Giải phóng tài nguyên trong onPause() hoặc onStop().

2. Sử dụng onSaveInstanceState() để lưu trạng thái:

```
override fun onSaveInstanceState(outState: Bundle) {
    super.onSaveInstanceState(outState)
    outState.putString("KEY", "value")
}
```

Sự kiện hệ thống ảnh hưởng đến Activity

Ví dụ các sự kiện:

- Gọi đến một ứng dụng khác.

- Quay màn hình.

- Bộ nhớ bị hạn chế.

Cách xử lý:

1. Sử dụng onSaveInstanceState() để lưu trạng thái.

2. Phục hồi trạng thái trong onCreate():

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    val value = savedInstanceState?.getString("KEY")
}
```

Phát triển ứng dụng tối ưu với các thành phần cơ bản

Nguyên tắc phát triển ứng dụng tối ưu:

- Sử dụng đúng thành phần cho mục đích cụ thể (Activity, Service, Broadcast Receiver, Content Provider).

- Tối ưu hóa vòng đời của Activity để tiết kiệm tài nguyên.

- Tái sử dụng mã nguồn qua các thành phần được chia nhỏ.

Ví dụ:

- Tách các chức năng như phát nhạc, nhận thông báo thành các Service riêng.

Mối quan hệ giữa các thành phần cơ bản

Sự phối hợp giữa các thành phần:

- Activity và Service: Giao diện điều khiển Service phát nhạc.

- Activity và Broadcast Receiver: Nhận thông báo hệ thống và cập nhật giao diện.

- Content Provider và các thành phần khác: Chia sẻ dữ liệu giữa ứng dụng.

Ví dụ:

- Ứng dụng đọc dữ liệu từ Content Provider và hiển thị trên Activity.

Các thách thức thường gặp

Thách thức trong lập trình Android:

- Quản lý tài nguyên hiệu quả.
- Đảm bảo giao diện hoạt động mượt mà.
- Xử lý vòng đời phức tạp của Activity.
- Tương thích với nhiều phiên bản Android.

Giải pháp:

- Sử dụng thư viện hỗ trợ như Android Jetpack.
- Kiểm thử ứng dụng trên nhiều thiết bị và phiên bản Android.

Bài tập thực hành

Nội dung thực hành:

1. Tạo ứng dụng với 2 Activity và chuyển dữ liệu qua Intent.

2. Triển khai Service phát nhạc trong nền.

3. Sử dụng Broadcast Receiver để hiển thị thông báo khi mạng bị ngắt.

Yêu cầu:

- Giao diện đơn giản nhưng trực quan.

- Hoạt động ổn định trên nhiều thiết bị.

Hướng dẫn giải bài tập

Hướng dẫn thực hành:

- Bài 1: Tạo Intent và truyền dữ liệu:

  1. Khai báo Intent và đính kèm dữ liệu bằng putExtra().

  2. Lấy dữ liệu bằng getStringExtra() ở Activity đích.

- Bài 2: Triển khai Service:

  1. Tạo lớp Service kế thừa từ Service.

  2. Khởi chạy Service bằng startService().

- Bài 3: Đăng ký Broadcast Receiver:

  1. Định nghĩa lớp kế thừa BroadcastReceiver.

  2. Đăng ký sự kiện trong AndroidManifest.xml.

Tổng kết nội dung buổi học

Nội dung đã học:

- Hiểu rõ các thành phần cơ bản: Activity, Service, Broadcast Receiver, Content Provider.

- Tìm hiểu vòng đời của Activity và cách quản lý hiệu quả.

- Sử dụng Intent để truyền dữ liệu và tương tác với hệ thống.

Phát triển:

- Nghiên cứu sâu hơn về Android Jetpack.

- Thực hành thêm với các bài tập nâng cao.

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Thank you!

THÁI NGUYÊN, THÁNG 10 NĂM 2024